

長庚資工軟硬體專題設計文件書

專題名稱：

AR Vision Link

執行期間： 114 年 9 月 至 116 年 1 月

指導教授： 吳齊人 教授

計畫參與人員：

B1229006 陳語嫻

B1229021 黃星昊

B1229031 黃柏瑞

B1229049 陳泓均

執行單位：長庚大學 資訊工程學系

中 華 民 國 2026 年 5 月 31 日

目錄

1. 系統簡介

- 1.1 規格目的
- 1.2 規格範圍
- 1.3 參考文件

2. 系統概述

- 2.1 系統目標
- 2.2 系統範圍
- 2.3 系統架構
- 2.4 軟/硬體建構項目需求概述
- 2.5 軟/硬體環境
- 2.6 一般限制

3. 設計內容

- 3.1 軟/硬體建構項目架構
 - 3.1.1 軟體模組架構圖
 - 3.1.2 部署架構圖
- 3.2 軟硬體組件設計說明
 - 3.2.1 主要類別設計
 - 3.2.2 任務對照表
 - 3.2.3 AI 分析流程設計
- 3.3 介面設計說明
 - 3.3.1 前端頁面介面
 - 3.3.2 REST API 介面規格
 - 3.3.3 資料庫介面
- 3.4 作業程序設計說明
 - 3.4.1 流程圖
 - 3.4.2 使用案例圖
 - 3.4.3 流程循序圖
 - 3.4.4 活動圖
- 3.5 輸入/輸出設計說明
 - 3.5.1 輸入設計
 - 3.5.2 輸出設計
 - 3.5.3 錯誤輸出設計
- 3.6 其他設計說明

3.6.1 安全性設計

3.6.2 Demo 模式設計

3.6.3 機器設定同步設計

3.6.4 即時通訊設計

4. 需求制設計之追溯與版本管理

4.1 需求追溯矩陣

4.2 文件版本

5. 附錄

5.1 訊息清單

5.2.檔案與程式對照表

5.3 檔案與報表對照表

5.4 實體關係圖

1.系統簡介

1.1 規格目的

本文件為「AR Vision Link」之設計文件書，依據課程設計文件格式整理，用以說明系統架構、模組規劃、介面設計與部署方式，以供後續開發、展示、維護與擴充參考。

1.2 規格範圍

範圍項目	說明
系統主題	webAR 臉部辨識與多人線上即時答題系統
核心模組	Vision 模組(臉部辨識+AR互動)、Link 模組(即時制答題互動)
使用環境	瀏覽器端操作，支援桌面裝置與具鏡頭之設備
系統介面	Web 前端介面、後端 API、資料庫與即時通訊模組
不包含範圍	行動 App、正式商業部署、完整教學平台後台

1.3 參考文件

技術項目	網址
React	https://react.dev/
React Router	https://reactrouter.com/
Socket.IO	https://socket.io/
REST API	https://expressjs.com/zh-tw/
Supabase	https://supabase.com/
WebRTC	https://webrtc.org/
ExpressTURN	https://www.expressturn.com/
Face API.js	https://justadudewhohacks.github.io/face-api.js/docs/index.html
CSS3	https://www.w3.org/TR/2001/WD-css3-roadmap-20010523/

GitHub Pages	https://pages.github.com/
Render	https://render.com/

2. 系統概述

2.1 系統目標

本系統旨在整合瀏覽器端的 AR 臉部追蹤與即時多人互動測驗機制，建置一套可直接於網頁執行的互動式 AR Quiz 平台。

透過結合 MediaPipe 臉部追蹤、即時分數顯示與教室情境設計，系統希望提升學習參與度與課堂互動性，讓教師能以主持人控制台管理測驗流程，學生則能透過個人裝置即時作答並在 AR 介面中看到自己的分數、排名與表現。

整體設計採前後端分離與模組化架構，作為後續導入個人化學習分析、題庫擴充以及與正式教學平台整合的基礎。

2.2 系統範圍

系統邊界內的功能與模組包括：

- Web 前端 AR 臉部追蹤與資訊顯示：以 MediaPipe 進行臉部關鍵點偵測，將玩家分數、作答結果與排名等資訊透過 AR 方式覆蓋在頭部位置上。
- 即時多人測驗與房間管理：支援建立測驗、建立房間、玩家加入與退出、題目推送、倒數計時與即時作答同步。
- 雲端資料儲存與成績管理：使用 Supabase PostgreSQL 儲存使用者資料、題庫、測驗場次、作答紀錄與分數，並提供基本的學習成效分析。
- 後端 API 與即時同步機制：後端負責題目與玩家資料的讀寫、測驗狀態維護，並透過 Supabase Realtime 將房間狀態與玩家作答結果即時推送至前端

系統邊界外的項目則不在本次實作範圍內，包括：

- 原生手機 App (iOS / Android) 客戶端開發。
- 與大型 LMS 或校務系統（如選課系統、成績系統）的深度整合。
- 與第三方單一登入 (SSO) 或學校帳號系統的正式綁定。

2.3 系統架構

本系統採用前後端分離的 Web 應用程式架構，前端以 React 建構單頁式應用（SPA），負責畫面呈現、AR 臉部追蹤、玩家互動與部份邏輯運算；後端則以 Node.js / Express.js 提供 REST API 與基礎伺服器環境，並搭配 Supabase 進行資料儲存與即時狀態同步。

在即時互動方面，系統利用 Supabase Realtime 監聽測驗房間、玩家紀錄與作答紀錄等資料表的變化，當主持人切換題目或玩家送出答案時，更新會即時推送至所有連線中的前端，達到多人同步作答與排行榜即時更新的效果。

整體設計偏向 heavy-client 模式：AR 視覺運算與畫面更新大多在使用者瀏覽器端完成，後端則聚焦於資料一致性、權限驗證及狀態同步，以利後續擴充題庫、分析服務或替換部署平台。

2.4 軟/硬體建構項目需求概述

類別	項目	需求說明
硬體	使用者端裝置	具備攝影機與可執行現代瀏覽器之桌上型電腦或筆記型電腦，建議配備穩定網路連線。
軟體	前端環境	React、React Router、JavaScript ES6+、HTML5、CSS3。
軟體	AR / 視覺模組	MediaPipe，用於臉部追蹤與臉部關鍵點（landmark）偵測與即時覆蓋。
軟體	資料儲存	Supabase PostgreSQL，提供資料庫、認證與即時資料同步服務。
軟體	後端伺服器	Node.js、Express.js，部署於 Render 雲端平台。
軟體	部署與靜態託管	前端可部署於 GitHub Pages 靜態網站託管服務。

2.5 軟/硬體環境

軟體環境（使用者端／施測端）

- 作業系統：Windows、macOS 或 Linux。
- 瀏覽器：支援現代 Web 標準之 Chrome、Edge 等 Chromium 系瀏覽器。

- 前端技術：React、React Router、JavaScript ES6 Modules、HTML5、CSS3。
- AR / AI 套件：MediaPipe（Face Tracking、Landmark Detection、即時圖層覆蓋）。

軟體環境（伺服器／後端）

- IDE 與開發工具：VS Code 等。
- 後端框架：Node.js、Express.js。
- 資料庫與後端服務：Supabase（PostgreSQL、認證、Realtime）。
- 部署與託管：
 - Frontend：GitHub Pages（或 Vercel / Netlify）。
 - Backend：Render（或 Railway / Fly.io 等支援 Node.js 的雲端平台）

硬體環境：

一般桌上型電腦或筆記型電腦，具備足夠效能以支援瀏覽器內的 MediaPipe 臉部追蹤與 React 應用執行。

- 內建或外接攝影機（Webcam），提供即時影像輸入。
- 穩定之網際網路連線，以確保 Supabase Realtime 與後端 API 通訊品質。

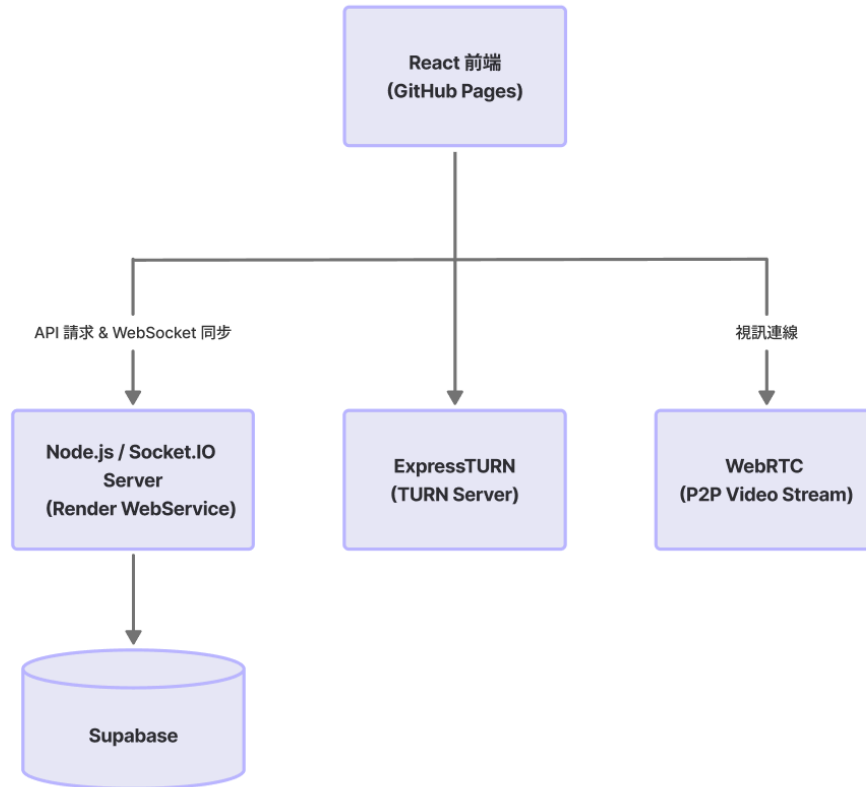
2.6 一般限制

- 系統須依賴瀏覽器對攝影機的存取授權與穩定的網路連線，若使用者拒絕權限或網路品質不佳，將無法正常進行 AR 顯示與多人同步測驗。
- 由於 AR 臉部追蹤與即時視覺運算皆在前端進行，其效能會受到裝置硬體規格、攝影機品質與光線條件影響，在低階裝置或弱光環境中可能出現幀率下降或偵測不穩定的情形。
- 即時同步功能仰賴 Supabase Realtime 及瀏覽器 WebSocket 連線品質，在高延遲或頻繁斷線的網路環境下，玩家畫面更新、倒數計時與排行榜顯示可能會出現延遲或不一致。
- 系統目前僅針對桌機及筆電瀏覽器進行設計與優化，對於部分行動裝置或不支援相關 Web API 的舊版瀏覽器，可能無法完整提供 AR 與多人互動功能。

3. 設計內容

3.1 軟/硬體建構項目架構

3.1.1 軟體模組架構圖



3.1.2 部署架構圖

- **Frontend**

- ├ Home
- ├ Register
- ├ FaceLogin
- ├ Profile
- ├ QuizHome
- ├ HostLobby
- ├ JoinQuiz
- ├ QuizGame
- ├ HostConsole

└─ Leaderboard

- Backend

- └─ REST API

- └─ Socket.IO Server

- └─ WebRTC Signaling

- └─ Supabase Service

3.2 軟硬體組件設計說明

3.2.1 主要類別設計 (Class Diagram)

- FrontendController：負責畫面切換與事件控制
- CameraManager：負責鏡頭啟動、影像擷取與 frame 更新
- FaceRecognitionEngine：負責臉部特徵擷取與辨識比對
- RoomManager：負責房間建立、狀態同步與事件廣播
- UserProfileService：負責註冊、資料讀寫與帳號管理

3.2.2 任務對照表 (TASK_MAP)

任務編號	任務名稱	負責模組
T01	使用者註冊	Register
T02	人臉登入	FaceLogin
T03	編輯個人資料	Profile
T04	建立測驗	CreateQuiz
T05	編輯測驗	ManageQuizzes
T06	建立房間	HostLobby
T07	加入房間	JoinQuiz
T08	開始遊戲	HostConsole

T09	玩家作答	QuizGame
T10	計算分數	Backend
T11	更新排行榜	Leaderboard
T12	WebRTC 視訊串流	HostConsole
T13	AR Overlay 顯示	TrackedPlayerVideo

3.2.3 AI 分析流程設計

各章節分析策略：

- 臉部偵測：以前端鏡頭影像取得人臉區域。
- 臉部追蹤：使用 MediaPipe FaceMesh 建立 3D 臉部關鍵點。
- 特徵辨識：以 face-api.js 建立特徵向量並進行比對。
- 身分確認：依比對結果完成登入、註冊或房間加入流程。

3.2.4 RAG 模組設計 (PDMS2Advisor)

本系統目前未規劃 RAG 模組。若後續擴充教學建議、答題紀錄分析或智慧提示，可於後端新增向量資料庫與知識檢索模組，作為未來版本延伸。

3.3 介面設計說明

3.3.1 前端頁面介面

頁面名稱	功能說明	輸入資料	輸出資料	對應檔案
Home	系統首頁與功能導覽	無	功能選單	Home.jsx
Register	使用者註冊	姓名、暱稱、個人介紹	使用者資料建立結果	Register.jsx
ReRegisterFace	重新註冊人臉	攝影機影像	新的人臉特徵資料	ReRegisterFace.jsx
FaceLogin	人臉登入	攝影機影像	登入成功或失敗	FaceLogin.jsx
Profile	個人資料管理	使用者資料	更新後個人資料	Profile.jsx

EditProfile	編輯個人資料	姓名、暱稱、介紹	更新結果	EditProfile.jsx
QuizHome	測驗功能首頁	無	測驗功能選單	QuizHome.jsx
CreateQuiz	建立測驗	測驗名稱、題目資料	測驗建立結果	CreateQuiz.jsx
ManageQuizzes	管理測驗	測驗資料	更新後測驗資料	ManageQuizzes.jsx
HostLobby	建立遊戲房間	測驗選擇	房號資訊	HostLobby.jsx
JoinQuiz	加入遊戲房間	房號	房間資訊	JoinQuiz.jsx
QuizGame	玩家作答頁面	答案選項	分數、作答結果、AR Overlay	QuizGame.jsx
HostConsole	主持人中控台	控制指令	玩家資訊、排行榜、視訊串流	HostConsole.jsx
Leaderboard	排行榜頁面	無	玩家排名與分數	Leaderboard.jsx
Camera	AR與人臉辨識測試頁面	攝影機影像	AR Overlay、人臉辨識結果	Camera.jsx

3.3.2 REST API 介面規格

施測端（前端 / Web 端）

- GET /：首頁
- POST /register：建立使用者資料
- POST /recognize：送出臉部特徵進行辨識
- POST /room/create：建立答題房間
- POST /room/join：加入房間
- POST /question/submit：送出答案

Mac Mini 伺服器端（改為一般後端伺服器）

- GET /health：檢查服務狀態
- GET /users：取得使用者資料
- GET /rooms：取得房間列表

管理端

- GET /admin：管理首頁
- GET /logs：事件紀錄檢視

- GET /stats：基本統計資訊

3.3.3 資料庫介面

users

欄位	型別	用途
id	bigint	使用者ID
name	varchar	名字
nickname	varchar	暱稱
description	text	自我簡介
extra_info	text	更多資訊
face_embedding	float8[]	人臉辨識特徵向量
avatar_url	text	頭像
created_at	timestampz	資料的創建時間

quizzes

欄位	型別	用途
quiz_id	bigint	測驗ID
host_id	bigint	主持人ID
title	varchar	測驗名稱
created_at	timestampz	測驗建立日期

questions

欄位	型別	用途
question_id	bigint	題目ID
quiz_id	bigint	測驗ID
question_text	text	問題

options	jsonb	選項
correct_answer	varchar	正確的答案
time_limit	integer	每題的時間限制

game_sessions

欄位	型別	用途
session_id	bigint	群組ID
<i>quiz_id</i>	bigint	測驗ID
room_code	varchar	房間號
current_question	integer	目前輪到哪一題
game_finished	boolean	遊戲結束與否
started_at	timestampz	開始時間
ended_at	timestampz	結束時間

player_answers

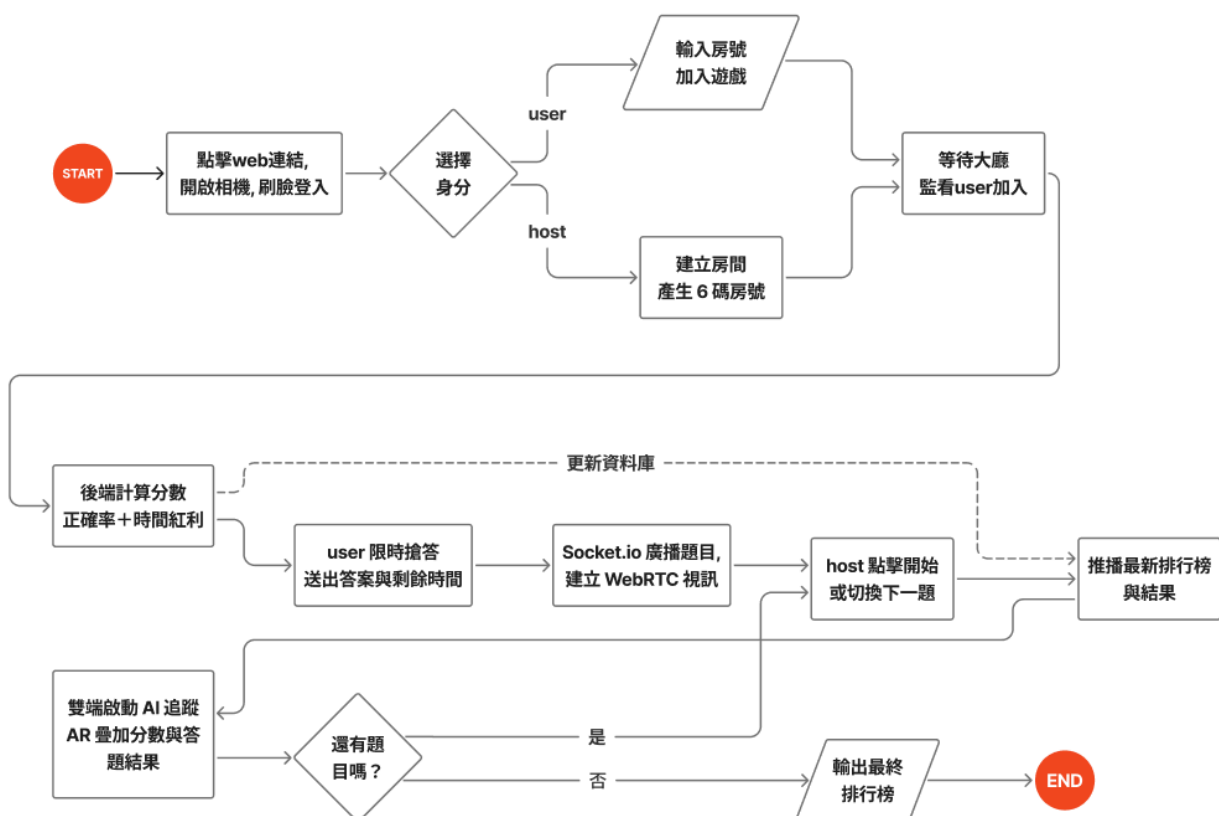
欄位	型別	用途
answer_id	bigint	ID
<i>session_id</i>	bigint	群組ID
<i>question_id</i>	bigint	題目ID
<i>user_id</i>	bigint	回答的玩家ID
answer	varchar	玩家回答的答案
is_correct	boolean	正確與否
score	integer	成績
answered_at	timestampz	答題時間

player_records

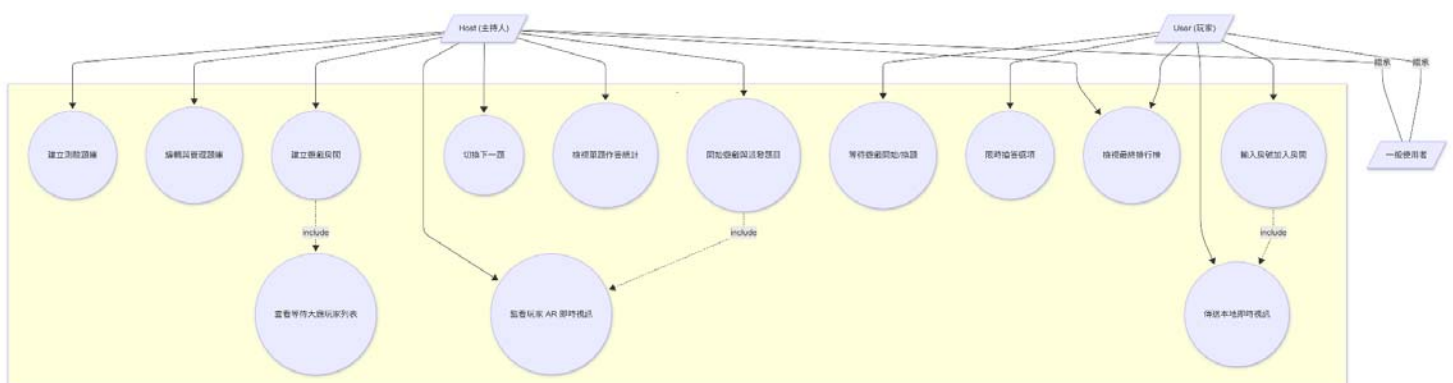
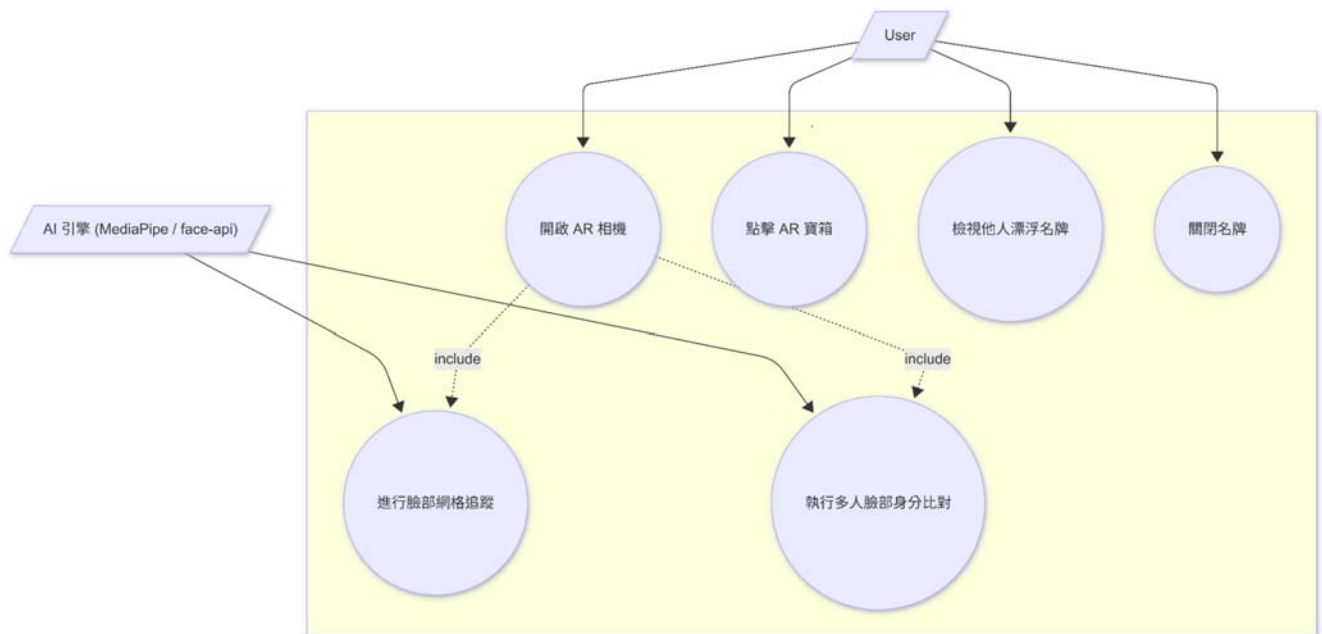
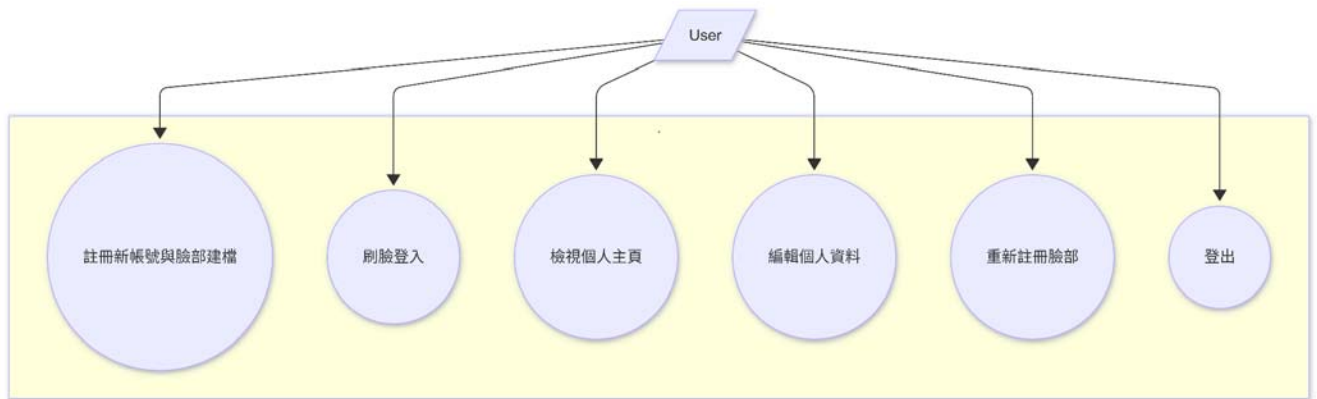
欄位	型別	用途
record_id	bigint	ID
session_id	bigint	群組ID
user_id	bigint	回答的玩家ID
score	integer	總成績
correct_count	integer	正確答題數
rank	integer	排名

3.4 作業程序設計說明

3.4.1 流程圖



3.4.2 使用案例圖 (Use Case Diagram)



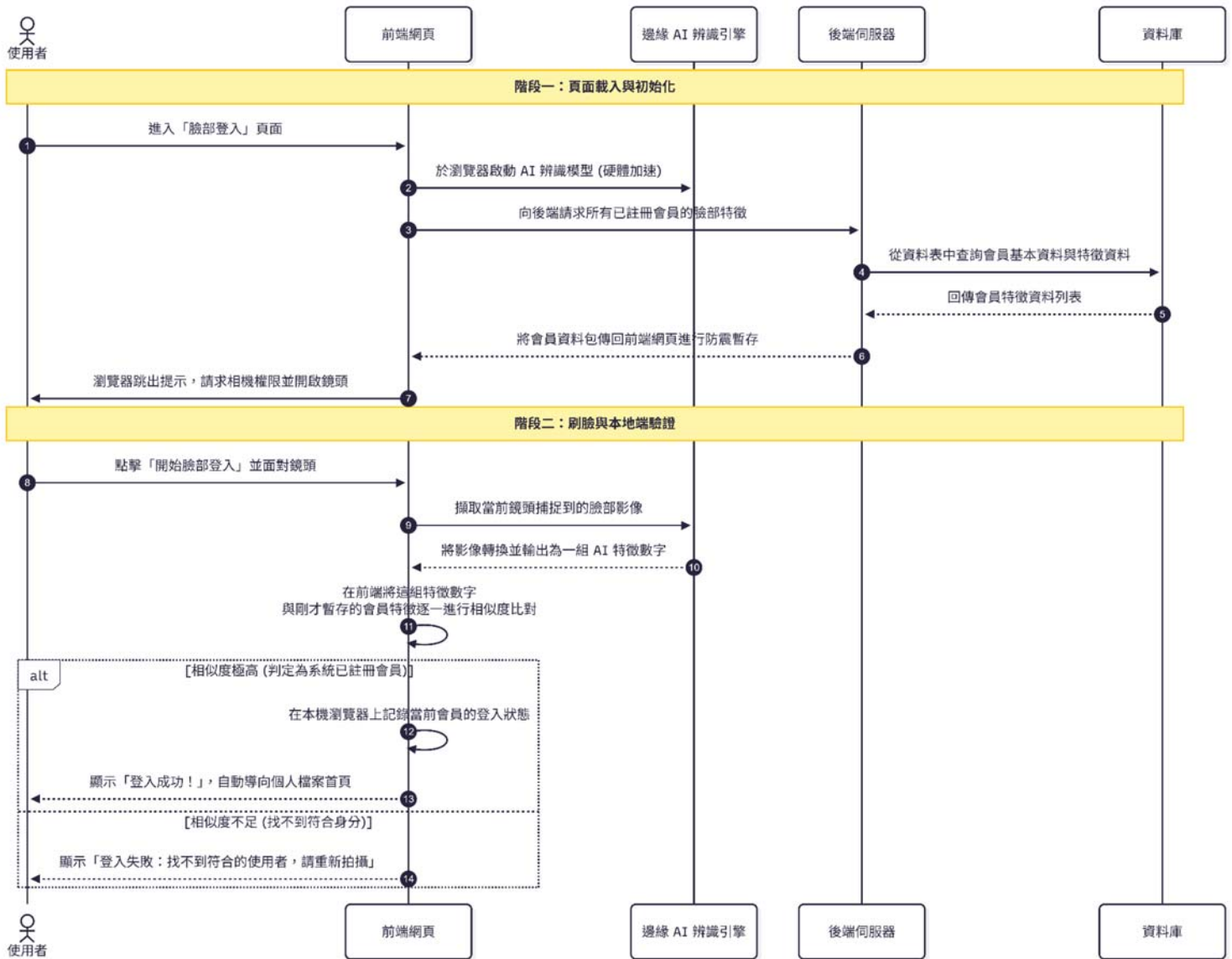
主要角色：

- 使用者(Host/User)
- 系統伺服器

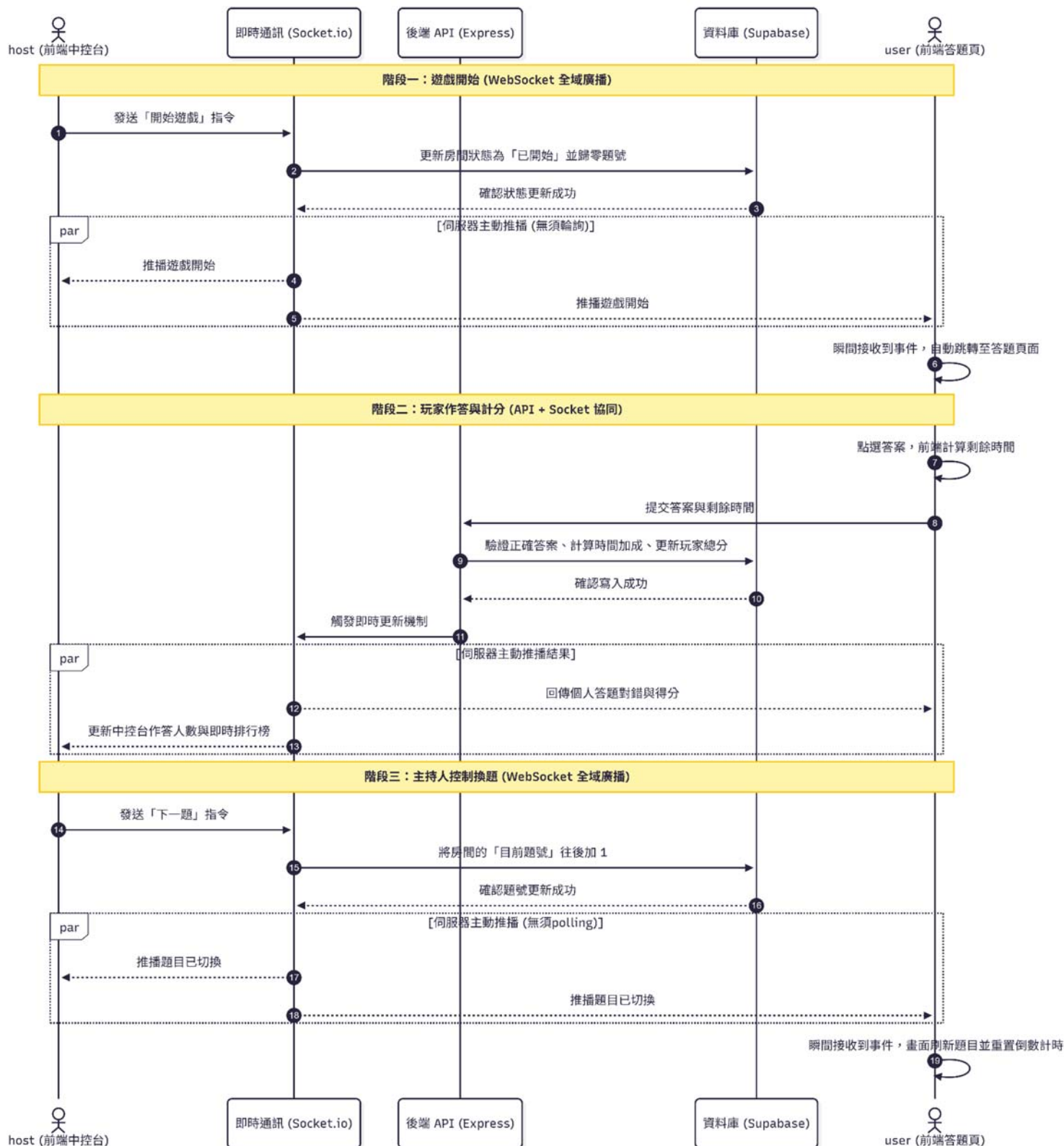
主要使用案例：

- 使用者註冊/登入/臉部辨識
- Host建立房間 / User加入房間 / 題目同步 / 作答送出

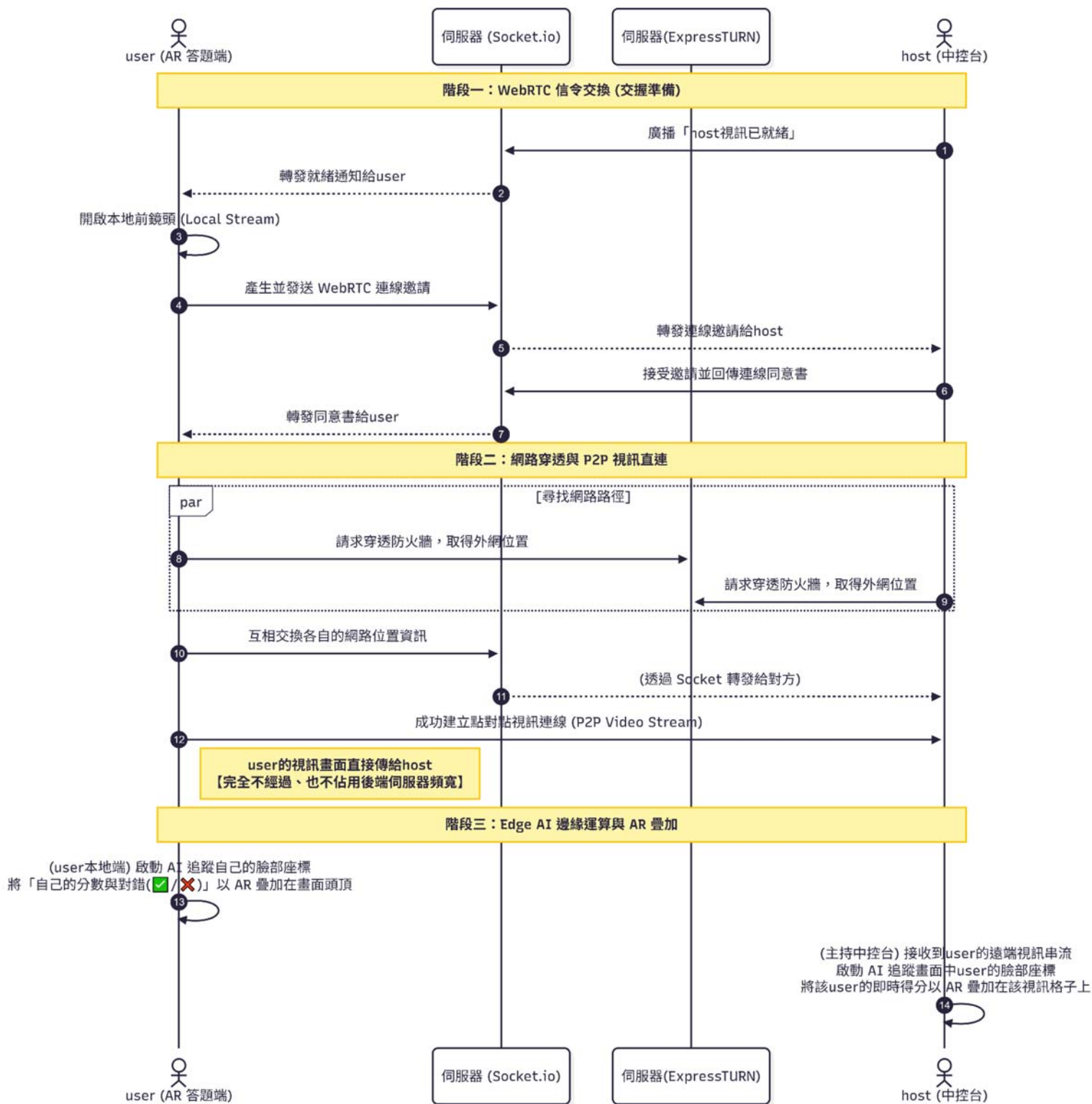
3.4.3 主要流程循序圖 (Sequence Diagram)



臉部辨識登入循序圖

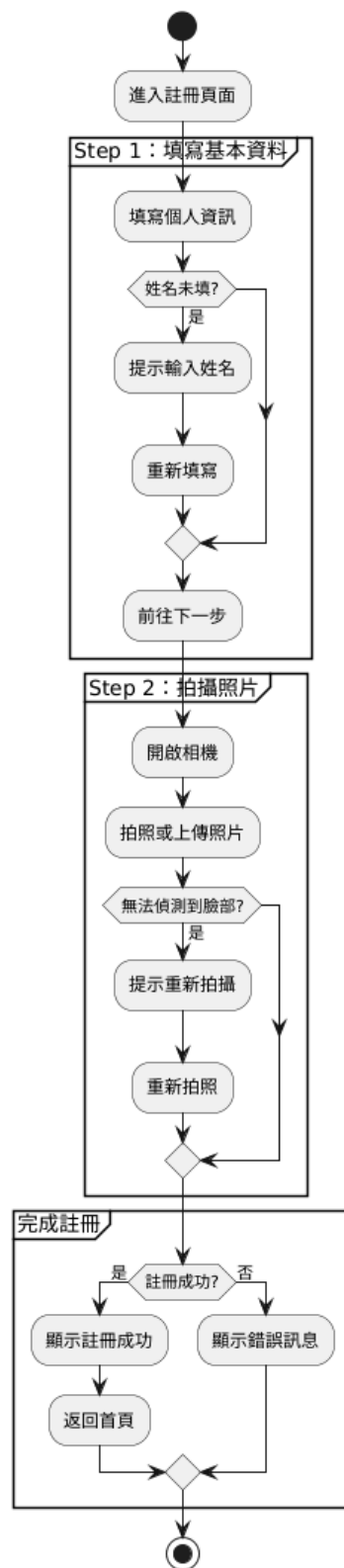


多人即時測驗同步循序圖

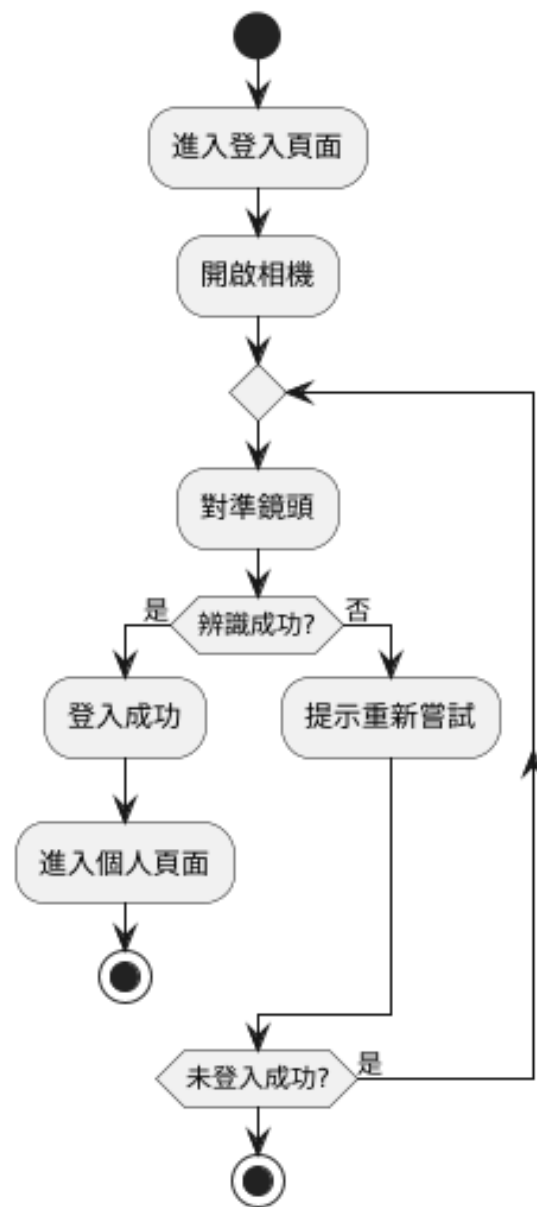


AR 影像互動循序圖

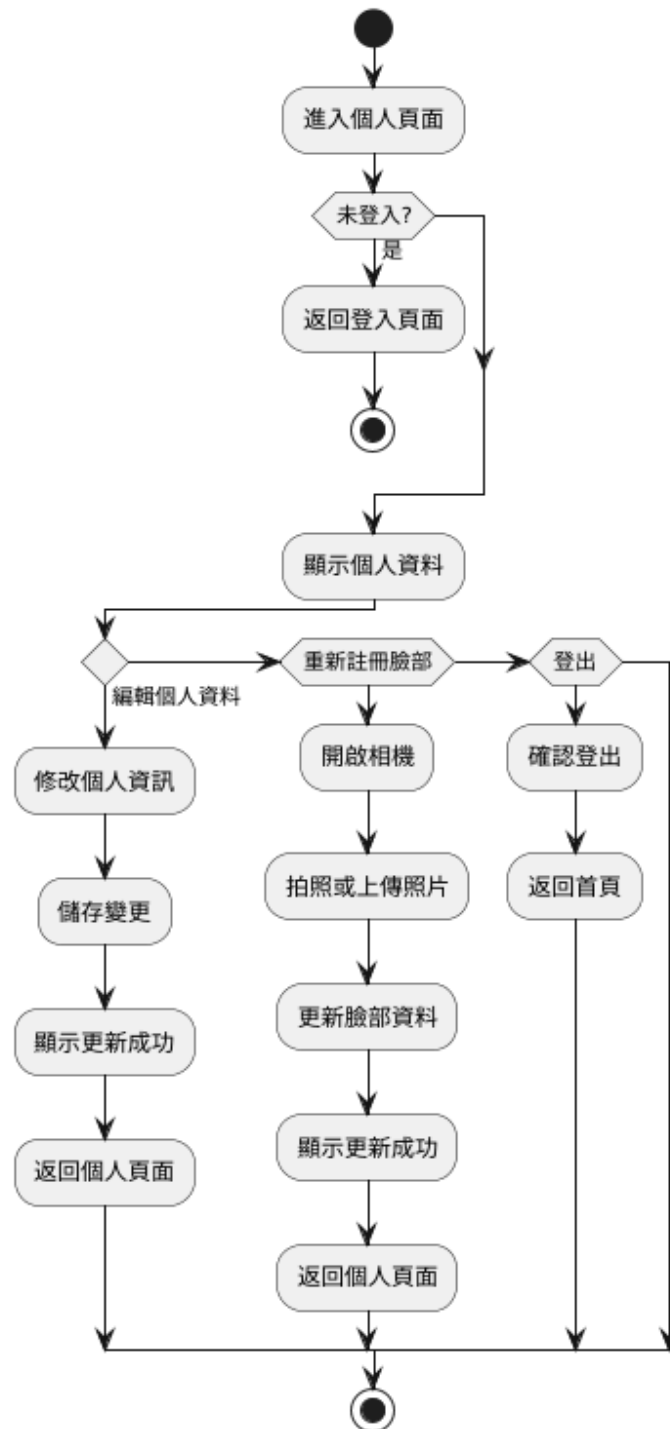
3.4.4 活動圖 (Activity Diagram)



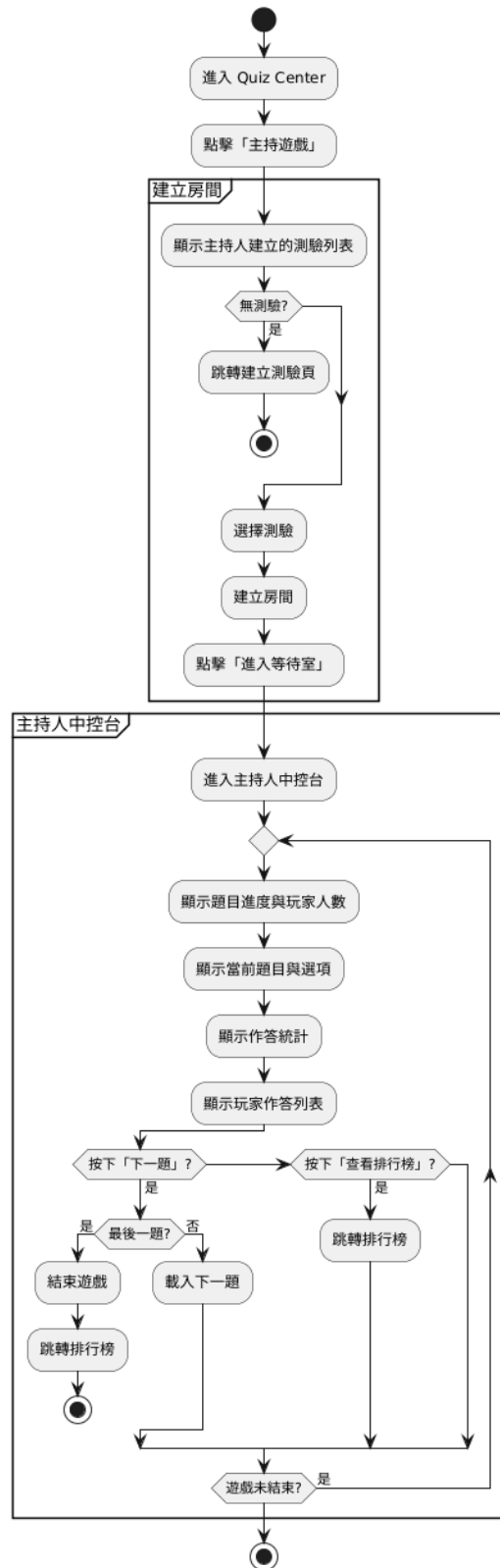
使用者註冊活動圖



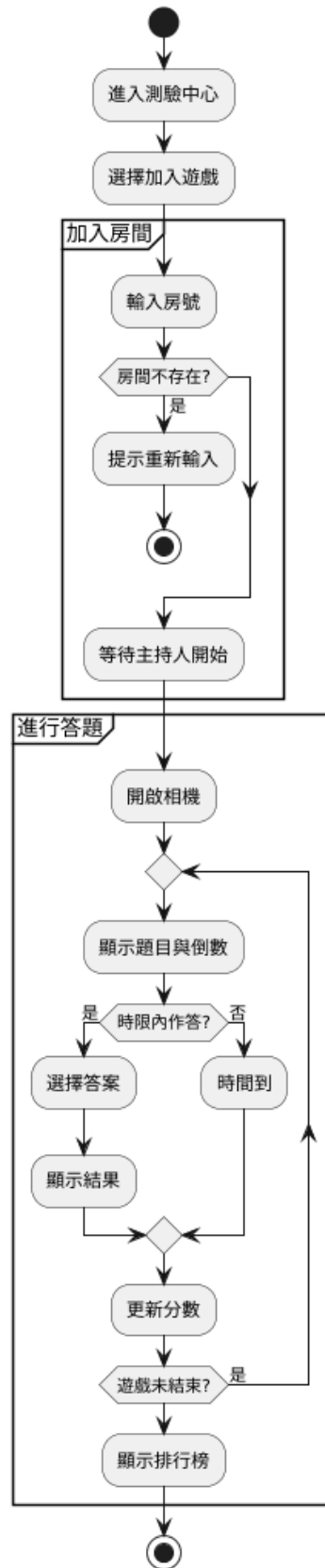
臉部登入活動圖



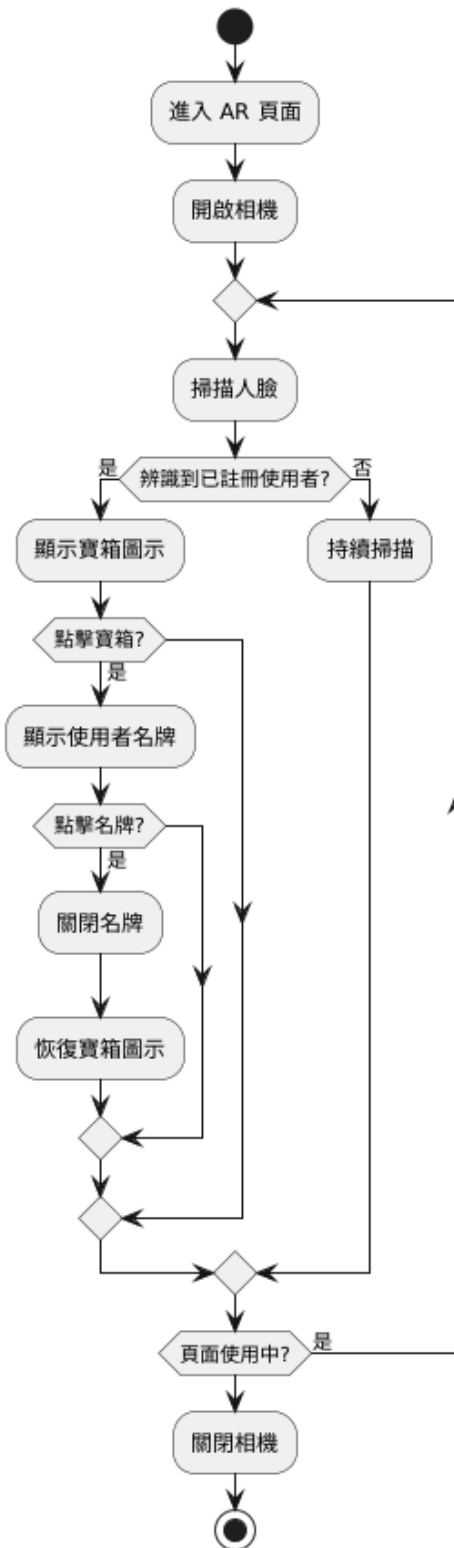
個人資料管理活動圖



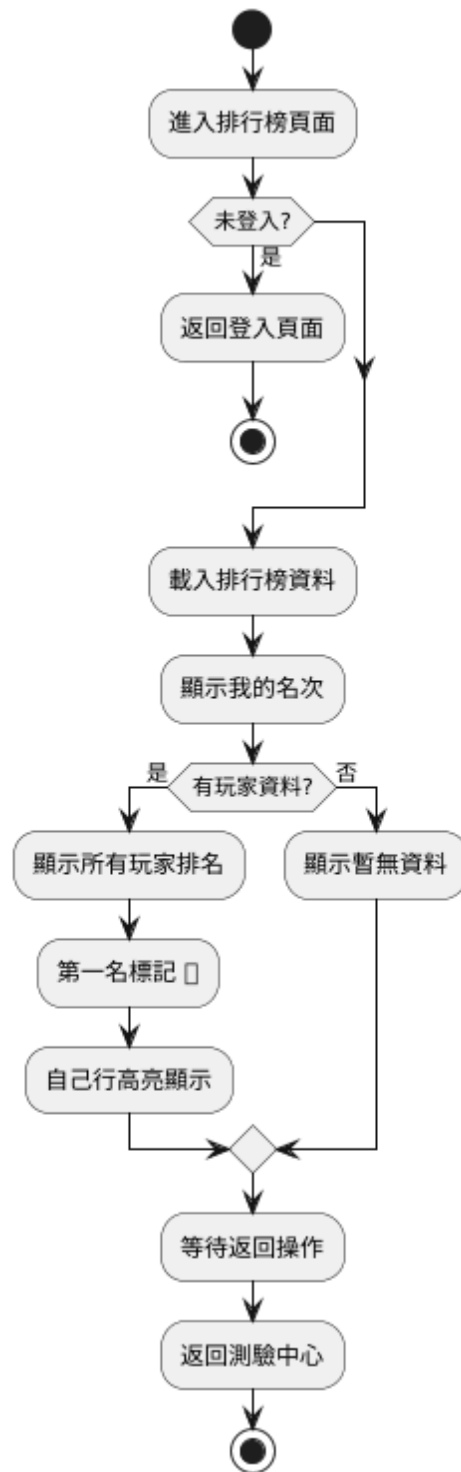
主持遊戲活動圖



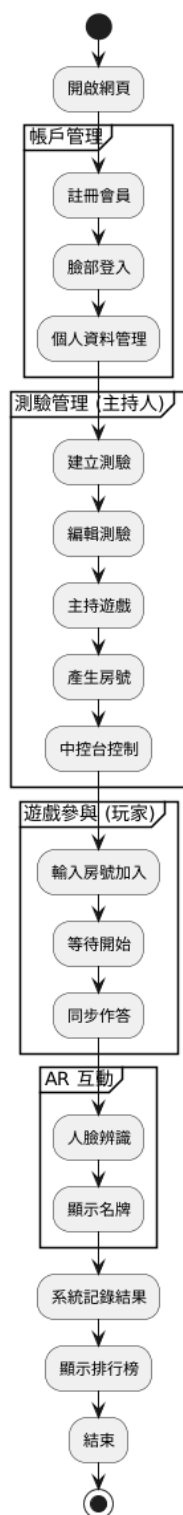
加入遊戲/作答活動圖



AR人臉辨識互動活動圖



排行榜活動圖



系統總活動圖

3.5 輸入/輸出設計說明

3.5.1 輸入設計

項目	型態	項目
房號	String	房號
題目	Text	題目
選項	JSON	選項
答案	String	答案

3.5.2 輸出設計

項目	型態	項目
分數	Integer	分數
排名	Integer	排名
作答結果	Boolean	作答結果
視訊串流	MediaStream	視訊串流

3.5.3 錯誤輸出設計

錯誤情境	錯誤輸出訊息	處理方式
未登入使用者進入系統	請先登入	導向 FaceLogin 頁面
房號不存在	找不到房間或房間已結束	提示玩家重新輸入房號
建立測驗失敗	建立測驗失敗	顯示錯誤並保留表單資料
玩家答案送出失敗	送出答案失敗	顯示提示並避免重複送出
Socket 連線錯誤	Socket 發生錯誤	顯示 alert 並停止當前操作

WebRTC 視訊連線失敗	等待玩家視訊...	Host 端顯示等待畫面
無法開啟鏡頭	無法開啟鏡頭，請允許瀏覽器相機權限	提醒使用者開啟相機權限
非主持人進入 HostConsole	沒有權限，只有主持人可以進入中控台	返回 AR Vision Link
遊戲已結束	遊戲已結束	導向排行榜頁面

3.6 其他設計說明

3.6.1 安全性設計

- 使用者資料與辨識資訊應透過受控 API 存取。
- 臉部特徵資料需避免明文暴露。
- WebSocket 與資料庫連線應設定基本存取控管。

3.6.2 Demo 模式設計

系統可於展示模式下以單機執行前端辨識流程，減少對完整後端服務之依賴，方便專題展示與功能驗證。

3.6.3 機器設定同步設計

若後續部署於多台裝置，可透過後端統一管理攝影機設定、房間規則與環境參數。

3.6.4 即時通訊設計

即時通訊模組以 WebSocket 為核心，負責房間狀態管理、題目同步、作答事件推播與排行榜更新。

4. 需求制設計之追溯與版本管理

4.1 需求追溯矩陣

需求編號	功能需求	對應模組
R01	人臉登入	FaceLogin
R02	建立測驗	CreateQuiz
R03	即時作答	QuizGame
R04	即時計分	Leaderboard
R05	視訊監控	HostConsole
R06	AR 顯示	TrackedPlayerVideo

4.2 文件版本

版本	日期	說明
v1.0	2026-05-29	依專題現況建立初版設計文件書

5. 附錄

5.1. 訊息清單

- 使用者登入成功
- 臉部辨識失敗
- 房間建立成功
- **Socket.IO Events**
 - join-session
 - game-started
 - question-changed
 - answer-submitted
 - leaderboard-updated
 - webrtc-offer
 - webrtc-answer
 - webrtc-ice-candidate

5.2. 檔案與程式對照表

檔案	功能
Home.jsx	首頁
Register.jsx	註冊
FaceLogin.jsx	人臉登入
QuizGame.jsx	玩家作答
HostConsole.jsx	主持人控制
TrackedPlayerVideo.jsx	AR Overlay
server.js	後端服務

5.3. 檔案與報表對照表

報表	來源
排行榜	player_records
題目統計	player_answers

5.4. 實體關係圖 (ER Diagram)

本系統可概念性分為三個主要實體：User、Room、QuizRecord。User 與 Room 為多對多關係，可透過房間成員表延伸；QuizRecord 與 User、Room、Question 形成作答紀錄關聯。

